Pet Care Planner Project Report

Introduction

Pet owners can use the planner to schedule procedures, add services, and see what needs attention. The application runs locally in a browser with a Java 21 backend. Its main journey runs from creating a care plan to completing the procedure and saving a downloadable summary.

User scenarios

Scenario	What the user does	Visible result
Create and configure	Enter a pet name, choose Bath and Brushing, and choose a date.	The saved plan shows the pet, procedure, extra, and 40 minutes.
Browse	Open the dashboard.	The page shows saved plans and upcoming or overdue reminders.
Edit and reschedule	Open a pending Saturday plan and select Next weekday.	The original plan now shows Monday as its due date.
Complete care	Select Mark complete on a due plan.	The plan remains in the list as completed and leaves Reminders.
Delete a plan	Select Delete and confirm.	The selected plan and its reminder are removed.
Export and reopen	Download the summary and restart the server.	A text file contains the results, and saved plans remain available.

Architecture

The browser form and Java HTTP handlers form the interface layer. Planner coordinates the use cases and relies on small interfaces for scheduling, duration, storage, and events. Plan groups a Pet, a Procedure, an optional Service, and the schedule and completion status.

Reference team roles are Student A and Student B. Before submission, enter your names, repository URL, and independently implemented contributions.

Design Patterns

Pattern	Problem it solves	Participants	How to demonstrate	Owner
Builder	Assemble and validate related fields together.	PlanBuilder, Plan, Planner	Save a complete plan and try a blank pet name.	A
Strategy	Select an interchangeable scheduling rule.	SchedulingStrategy, ExactDate, NextWeekday	Switch the same Saturday between exact date and weekday.	A
Decorator	Add an optional service to duration calculation.	Care, Procedure, WithService	Compare Bath at 30 minutes and Bath with Brushing at 40.	A
Adapter	Bridge a typed planner store and a byte-based file.	PlanStore, PropertiesStoreAdapter, PropertiesFile	Save, restart, and reopen the plan.	B
Observer	React to plan changes by updating reminders.	PlanEvents, PlanObserver, ReminderBoard	Complete an overdue plan and watch its reminder disappear.	B
Facade	Give the HTTP layer simple use-case entry points.	Planner, HTTP routes, planning and storage components	Trace POST /save through Planner.configure.	B

Selected Java excerpts

Strategy defines one method so the facade can call either algorithm through the same interface. Decorator uses the Care interface on both sides of the wrapper.

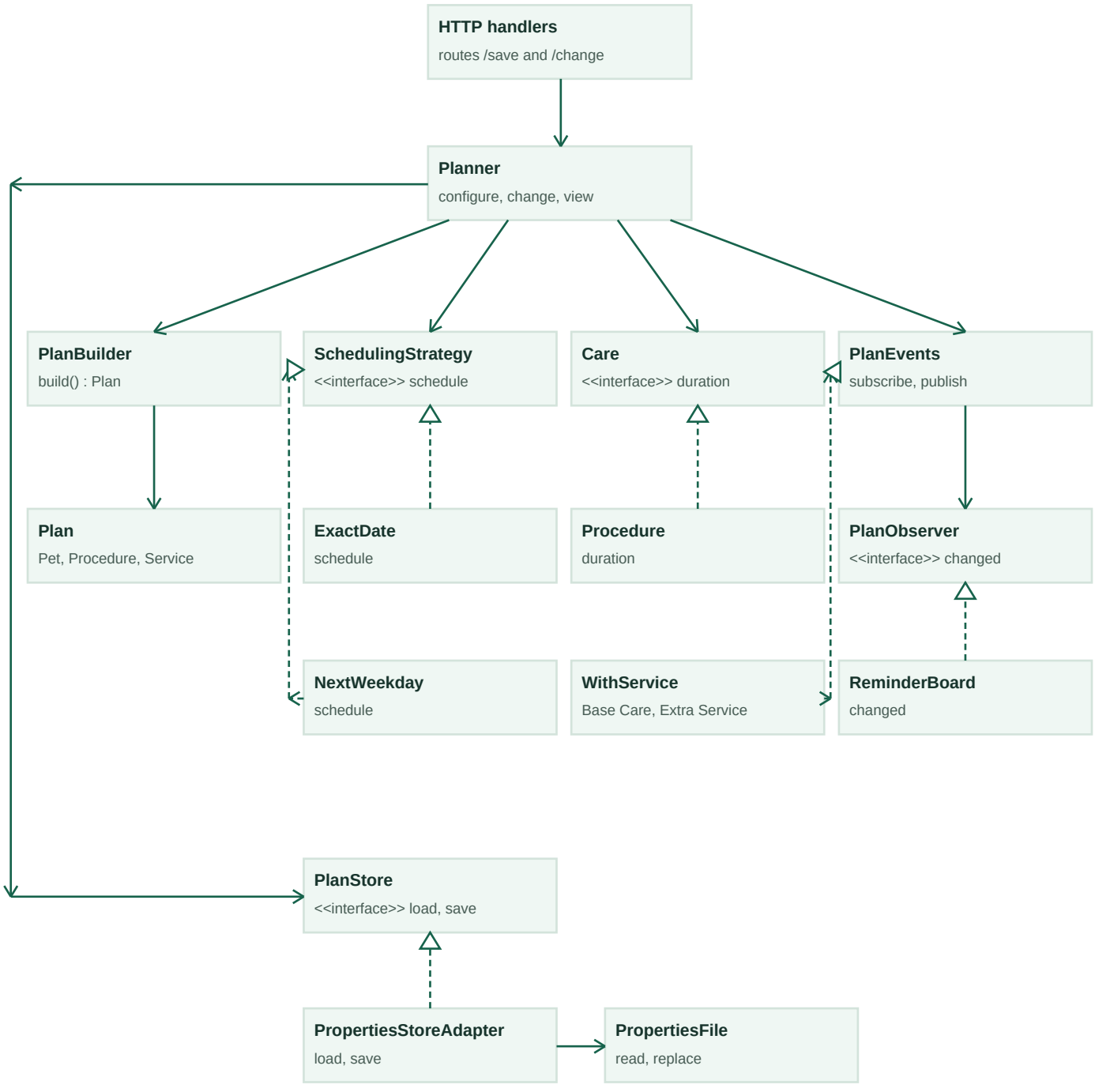
```
interface SchedulingStrategy {
    LocalDate schedule(LocalDate date);
}

public int duration() {
    return base.duration() + extra.minutes();
}
```

A successful write changes in-memory state before the facade publishes the event. A failed write returns before either step.

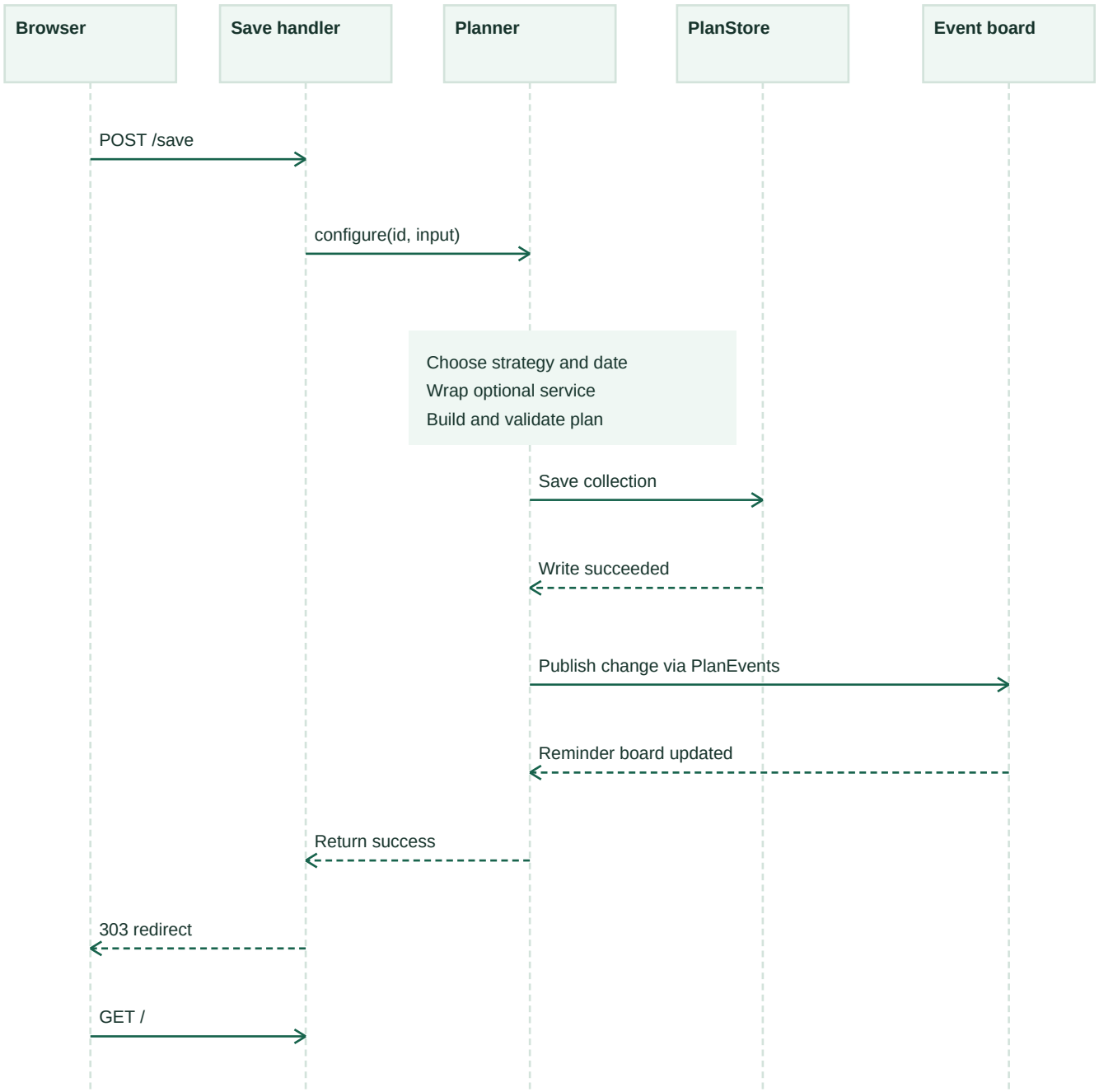
```
store.save(updated);
plans = updated;
events.publish(event);
```

UML Class Diagram



Solid arrows show use or a held reference. Dashed arrows with open triangles show Java interface realization. The Plan box lists its related domain entities.

UML Save Sequence



The redirected request reads the latest plans and reminders and renders the dashboard. If validation or saving fails, the handler renders an error and the proposed update stays out of the plan list and reminder board.

Application Screenshots

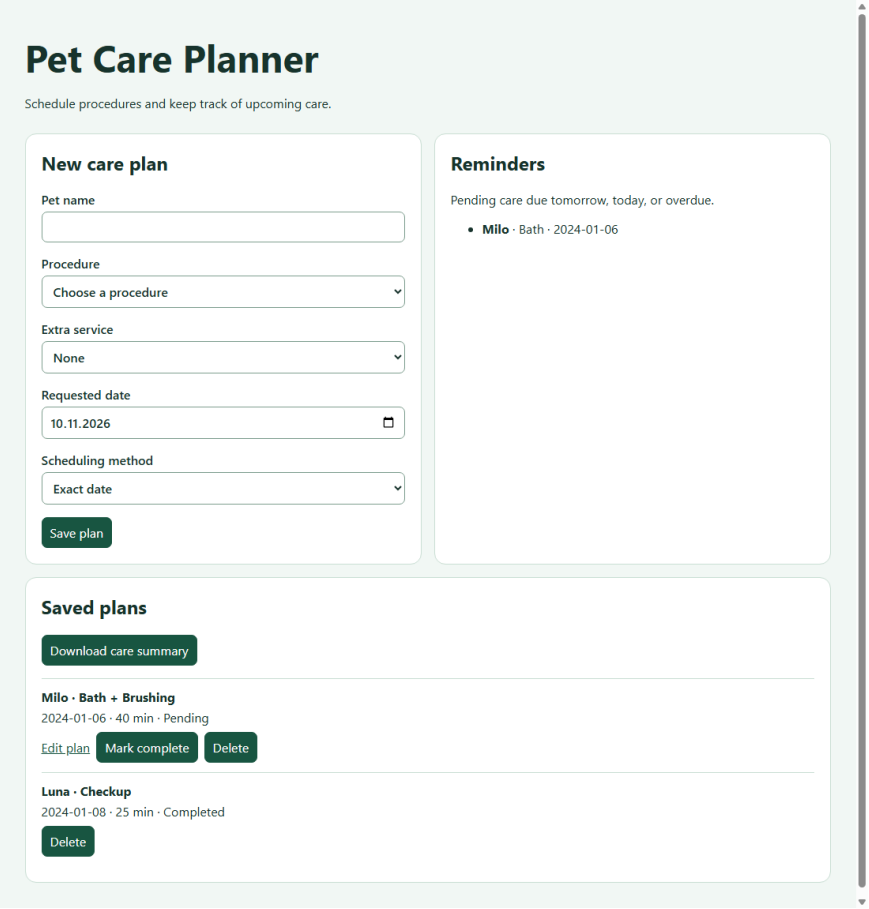


Figure 1. The dashboard with the new-plan form, the overdue Milo reminder, and the sample plans.

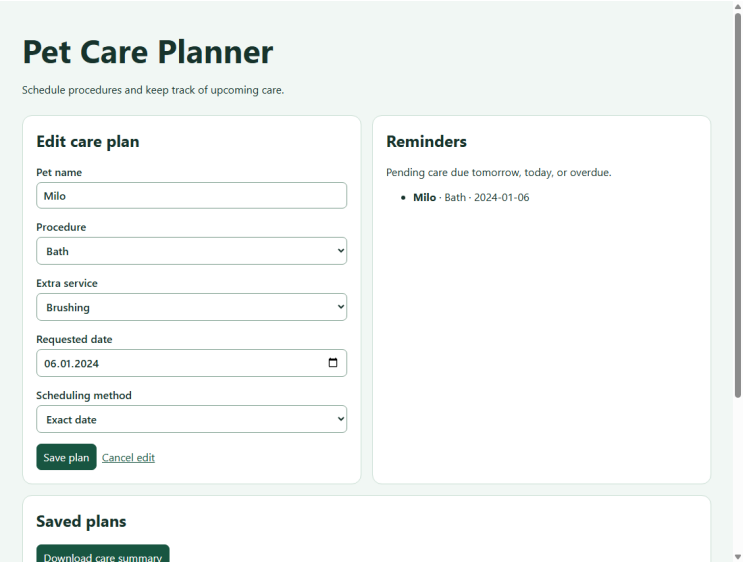


Figure 2. Editing Milo opens a form populated from the saved plan.

Testing and Contributions

Inputs	Expected result	Actual result in reference
Milo, Bath, Brushing, Exact date, 2024-01-06	One 40-minute plan due January 6.	Passed. One plan, 40 minutes, January 6.
Edit to Next weekday and remove Brushing	Same plan due January 8, 30 minutes.	Passed. Monday January 8, 30 minutes.
Reopen saved properties file on January 7	Saved plan and January 8 reminder load.	Passed. One saved plan and one reminder.
Complete the pending plan	Completed status, reminder removed, summary updated.	Passed. Status and summary updated, zero reminders.
Pet name containing only spaces	Reject the name and leave saved plans alone.	Passed. Name error and unchanged plan count.
Date 2024-02-30	Reject the invalid date.	Passed. Date error and unchanged plan count.
Service value Unknown	Reject the unavailable service.	Passed. Service error and unchanged plan count.
Edit a completed plan	Reject the edit.	Passed. Completed plan error.
Delete the plan	Plan no longer appears.	Passed. No plans remain.
POST form, then GET / and /summary	Redirect, saved plan and reminder in HTML, text attachment.	Passed. HTTP 303, HTTP 200, and expected attachment content.

Individual contributions

Proposed split. Student A owns Builder, Strategy, Decorator, and business tests. Student B owns Adapter, Observer, Facade, and interface integration. Replace this split with actual contributions and commit or pull request links.

Conclusion and future work

The integrated app takes a plan through configuration, scheduling, persistence, reminders, completion, and export. Recurring schedules, multiple extras, and a shared database are possible future additions.

AI disclosure. OpenAI Codex generated this reference implementation, test suite, and report draft from the course brief. Update this disclosure to reflect how your team used it and list any other sources before submission.